

بازاندیشی در مورد طراحی اولیه

آیا اگر از ابتدا نیازمندی «پشتیبانی از سامانه نظارت هوشمند آزمون (Online Exam Proctoring)» مشخص بود، طراحی متفاوتی انجام می‌شد؟

یکی از اهداف اصلی فاز مدیریت تغییر، ارزیابی میزان انعطاف‌پذیری معماری نرم‌افزار در برابر نیازمندی‌های جدید است. در این پروژه، نیازمندی جدید با عنوان «پشتیبانی از سامانه نظارت هوشمند آزمون (Online Exam Proctoring)» به عنوان سناریوی تغییر انتخاب شد. این قابلیت با هدف افزایش امنیت آزمون‌های آنلاین، پایش رفتار دانش‌آموزان در زمان برگزاری آزمون و ثبت خودکار رویدادهای مشکوک به تقلب تعریف گردید.

بازاندیشی در طراحی اولیه با این پرسش آغاز می‌شود که اگر این نیازمندی از ابتدای پروژه مشخص بود، آیا معماری و طراحی سامانه به شکل متفاوتی انجام می‌شد یا خیر.

ارزیابی طراحی اولیه

معماری اولیه سامانه آموزش آنلاین بر اساس Layered Architecture و با بهره‌گیری از الگوهای Repository، Event-Driven Architecture، Pattern، Strategy Pattern، Decorator Pattern، Dependency Injection طراحی و پیاده‌سازی شد. این معماری جداسازی مناسبی بین لایه‌های Domain، Repository، Service، Event و Controller ایجاد کرده و وابستگی میان اجزای سیستم را به حداقل رسانده است.

در زمان طراحی اولیه، ماژول آزمون تنها مسئول مدیریت فرآیند برگزاری آزمون، ثبت پاسخ‌ها، محاسبه نمره و اعلام نتیجه بود و فرض بر این بود که محیط برگزاری آزمون قابل اعتماد است. بنابراین هیچ مؤلفه مستقلی برای پایش رفتار کاربران، تحلیل رویدادهای امنیتی یا مدیریت تخلفات آزمون در نظر گرفته نشده بود.

تغییراتی که در صورت اطلاع از نیازمندی جدید از ابتدا اعمال می‌شد

در صورتی که نیازمندی «سامانه نظارت هوشمند آزمون» از ابتدای پروژه مشخص بود، معماری سیستم با در نظر گرفتن یک زیرسامانه مستقل برای مدیریت فرآیند نظارت طراحی می‌شد. این زیرسامانه می‌توانست با عنوان Online Proctoring Module یا Assessment Monitoring Module در کنار سایر ماژول‌های اصلی سامانه قرار گیرد و مسئولیت دریافت، تحلیل و مدیریت تمامی رویدادهای مرتبط با نظارت بر آزمون را بر عهده داشته باشد.

در این حالت، مسئولیت سرویس آزمون تنها به مدیریت فرآیند آموزشی محدود می‌شد و منطق مربوط به تشخیص تخلف، تحلیل رفتار دانش‌آموز، ثبت هشدارها و تصمیم‌گیری درباره ادامه یا خاتمه آزمون به سرویس مستقل ProctoringService واگذار می‌گردید. این تفکیک مسئولیت‌ها علاوه بر رعایت بهتر اصل Single Responsibility Principle (SRP)، توسعه و نگهداری سیستم را نیز ساده‌تر می‌کرد.

همچنین مدل دامنه سیستم از ابتدا با موجودیت‌های تخصصی‌تری مانند ExamViolation، ProctoringEvent و ViolationReport طراحی می‌شد تا اطلاعات مربوط به تخلفات، هشدارها، زمان وقوع، شدت رویداد و وضعیت رسیدگی به صورت ساخت‌یافته در سیستم مدیریت شوند. وجود این موجودیت‌ها امکان توسعه قابلیت‌هایی نظیر تحلیل آماری تخلفات، گزارش‌گیری مدیریتی و ارزیابی رفتار کاربران را نیز فراهم می‌کرد.

تأثیر بر معماری رویدادمحور

از آنجا که پروژه از معماری Event-Driven استفاده می‌کند، افزودن قابلیت نظارت هوشمند بدون تغییر اساسی در ساختار فعلی امکان‌پذیر است. با این حال، در صورت اطلاع از این نیازمندی از ابتدای پروژه، مجموعه‌ای از رویدادهای تخصصی آزمون از همان ابتدا طراحی می‌شدند. نمونه‌ای از این رویدادها عبارت‌اند از:

- ExamStarted
- TabSwitched
- CameraDisconnected
- NetworkInterrupted
- SuspiciousActivityDetected
- ExamViolationReported

در این معماری، EventPublisher صرفاً مسئول انتشار رویدادها باقی می‌ماند و Listenerهای مختلف، از جمله ماژول اعلان‌ها، سیستم ثبت وقایع، سرویس نظارت و بخش گزارش‌گیری، هر یک وظایف اختصاصی خود را بدون ایجاد وابستگی مستقیم به سایر اجزا انجام می‌دادند. این ساختار علاوه بر افزایش توسعه‌پذیری، انطباق کاملی با اصل Open/Closed Principle (OCP) دارد؛ زیرا افزودن رویدادها یا Listenerهای جدید بدون تغییر در هسته سیستم امکان‌پذیر خواهد بود.

ارزیابی کیفیت معماری اولیه

اگرچه نیازمندی جدید باعث افزودن چند کلاس، سرویس و رویداد جدید به سیستم می‌شود، اما بررسی انجام‌شده نشان می‌دهد که معماری اولیه پروژه از انعطاف‌پذیری مناسبی برخوردار بوده است. استفاده از معماری لایه‌ای، جداسازی مسئولیت‌ها، الگوهای طراحی و تزریق وابستگی موجب شده است که توسعه قابلیت جدید بدون نیاز به بازطراحی اساسی هسته سامانه امکان‌پذیر باشد.

در واقع، بیشتر تغییرات موردنیاز در قالب افزودن مؤلفه‌های جدید انجام می‌شوند و تغییر محسوسی در منطق موجود سرویس‌ها، کنترلرها و Repositoryها ایجاد نمی‌شود. این موضوع بیانگر آن است که تصمیمات معماری اتخاذشده در مراحل اولیه پروژه، قابلیت پاسخ‌گویی به تغییرات آینده را تا حد زیادی فراهم کرده‌اند.

جمع‌بندی

بررسی انجام‌شده نشان می‌دهد که اگر نیازمندی «پشتیبانی از سامانه نظارت هوشمند آزمون» از ابتدای پروژه مشخص بود، طراحی اولیه با پیش‌بینی یک زیرسامانه مستقل برای مدیریت فرآیند نظارت، مدل دامنه تخصصی‌تر و مجموعه‌ای از سرویس‌ها و رویدادهای اختصاصی انجام می‌شد. با این وجود، معماری انتخاب‌شده در پروژه به دلیل بهره‌گیری از اصول مهندسی نرم‌افزار و الگوهای طراحی مناسب، امکان اعمال این تغییر را بدون نیاز به بازنویسی ساختار اصلی سیستم فراهم کرده است. این نتیجه نشان می‌دهد که طراحی اولیه سامانه از انعطاف‌پذیری، توسعه‌پذیری و قابلیت نگهداری مناسبی برخوردار بوده و توانسته است تغییرات جدید را با حداقل هزینه و ریسک پشتیبانی کند.